

Arquitectura e Ingeniería de Computadores

Grado de Ingeniería del Software



TEMA 4. JERARQUÍA DE MEMORIA (PARTE 2)

Ángel Serrano Sánchez de León

Óscar David Robles Sánchez

Sofía Bayona Beriso

David Miraut Andrés

Luis Rincón Córcoles

CONTENIDOS

- Introducción
- Elementos de diseño
- Aumento del rendimiento: memoria TLB

INTRODUCCIÓN

- El concepto de memoria virtual surge porque los computadores tienen **varios programas** ejecutándose a la vez y, por tanto, dichos programas y sus datos deben estar cargados en la memoria principal.
- La memoria principal (pequeña y limitada) es ampliada o expandida mediante el almacenamiento secundario (disco): **zona de intercambio o swap**.
- La memoria principal actúa como una “caché” para el almacenamiento secundario: esto es la **memoria virtual**.
- El funcionamiento está garantizado gracias a la **proximidad referencial** de los programas, igual que en la memoria caché.

3

- Todo lo estudiado hasta ahora presupone que solo tenemos un solo proceso activo.
 - Los procesos son programas puestos en ejecución por el sistema operativo, junto con su código, bibliotecas de funciones, datos, pila, ficheros proyectados en memoria, subrutinas, etc.
- Los computadores modernos permiten tener varios procesos compartiendo el procesador y, por tanto, también la memoria.
 - En el momento de compilar no podemos saber con qué otros programas vamos a compartir la máquina.
 - Además, esto cambia dinámicamente a medida que los programas se ejecutan.
- Se dice que un computador emplea memoria real cuando el mapa de memoria está soportado exclusivamente por memoria principal y caché.
- Por el contrario, un computador emplea memoria virtual cuando el mapa de memoria está soportado por una mezcla de memoria principal y de disco duro.
 - En este caso, hay que distinguir entre direcciones de memoria virtuales, generadas por el procesador, y direcciones de memoria reales, necesarias para acceder a la memoria principal.
- Cuando varios programas se ejecutan al mismo tiempo en una máquina, la memoria total requerida para ellos puede ser mayor que el espacio de memoria principal libre en la máquina, aunque solo una parte de esta memoria se use activamente en cada instante de tiempo.

INTRODUCCIÓN

- La memoria virtual permite **traducir** el espacio de direcciones disponible (virtuales) a direcciones físicas (memoria principal o disco duro). Esta traducción se hace con una mezcla de circuitería y software.
- El proceso de traducción **asegura la protección** del espacio de direcciones de un programa de los demás programas en ejecución y también favorece la **reubicación** (un programa puede cargarse en cualquier parte de la memoria principal).

- Hay que proteger el espacio de memoria de cada programa, asegurando que cada uno lea y escriba solamente en las partes de la memoria que tenga asignadas.
- Además, sería bueno poder compilar cada programa en su propio espacio de direcciones, una parte de direcciones de memoria solo accesibles para ese programa.

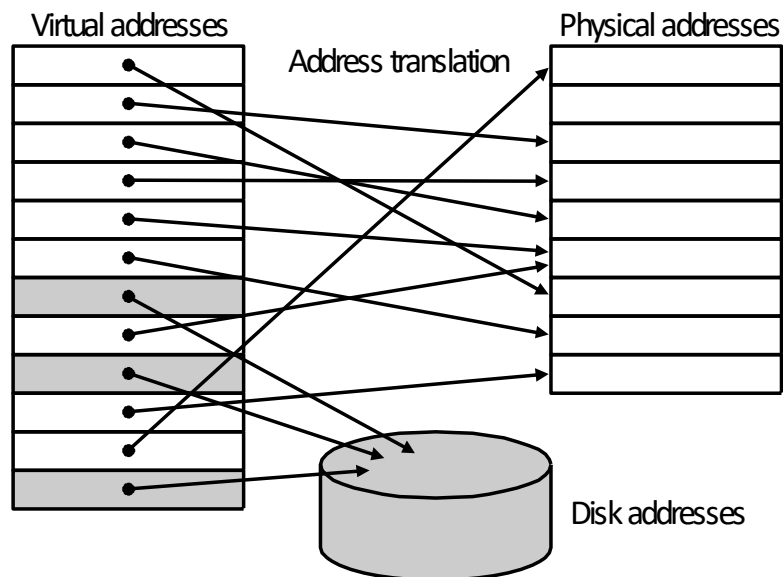
INTRODUCCIÓN

- La memoria virtual gestiona automáticamente los dos niveles de la jerarquía de memorias implicados:
 - La memoria principal (también llamada **memoria física** para distinguirla de la virtual).
 - Almacenamiento masivo (**disco**).
- Terminología distinta respecto de memoria caché:
 - Un bloque en memoria virtual se llama **página**, que será asignado a un bloque de memoria principal llamado **marco de página** (mismo tamaño).
 - Un fallo en memoria principal se llama **fallo de página**.
- Con memoria virtual, la CPU produce direcciones virtuales, que se traducen a direcciones físicas.

5

- Similitudes y diferencias entre memoria virtual y caché:
 - En ambos casos se cumplen los principios de localidad temporal y espacial (proximidad de referencias).
 - Basta con tener una pequeña parte de toda la información en el nivel de memoria más rápido para que todo el sistema funcione correctamente.
 - La transferencia de datos entre los dos niveles de memoria se hace por bloques.
 - La memoria virtual se divide en bloques del mismo tamaño llamados "páginas".
 - Hay una diferencia desproporcionada entre los tiempos de acceso de la memoria principal y el disco duro, mientras que esta diferencia no es tan acusada entre caché y memoria principal.
 - La penalización por fallo de página es muy costosa, con lo que se debe conseguir que la tasa de aciertos de la memoria virtual sea muchísimo mayor que la de la memoria caché.
 - En ambos casos hay que especificar la función de correspondencia, la identificación de la información almacenada, la política de reemplazo, la de escritura, etc.
 - Muchas de las cosas que ya sabemos de la caché se puede aplicar a la memoria virtual.
 - ¿Quién se encarga de gestionar cada parte?
 - La memoria caché está implementada enteramente por hardware.
 - En el caso de la memoria virtual, buena parte de su gestión se realiza por software.

INTRODUCCIÓN



- El procesador genera direcciones virtuales:
 - Pueden corresponder a datos ubicados en la memoria principal o en el disco duro.
- El espacio de direcciones virtuales está dividido en páginas (llamadas páginas virtuales).
- Asimismo, el espacio de direcciones físicas está dividido también en páginas del mismo tamaño (llamadas páginas físicas o marcos).
- Mediante un proceso de traducción de direcciones, se convierte la dirección virtual en dirección física (de memoria principal), o bien, en una posición del disco duro.
- Dos o más páginas virtuales pueden apuntar al mismo marco físico, por ejemplo, cuando dos programas comparten código o datos.
- Las páginas virtuales marcadas en gris no están cargadas en un marco, con lo que se encuentran almacenadas en el disco duro.
 - Necesitaremos un bit de validez o de presencia que nos indique si una página virtual está cargada en un marco.
- **IMPORTANTE: la paginación es invisible al programador/a.**

CONTENIDOS

- Introducción
- Elementos de diseño
- Aumento del rendimiento: memoria TLB

ELEMENTOS DE DISEÑO DE LA MEMORIA VIRTUAL

- Segmentación o paginación
- Traducción de direcciones (identificación)
- Integración con la caché
- Otros aspectos

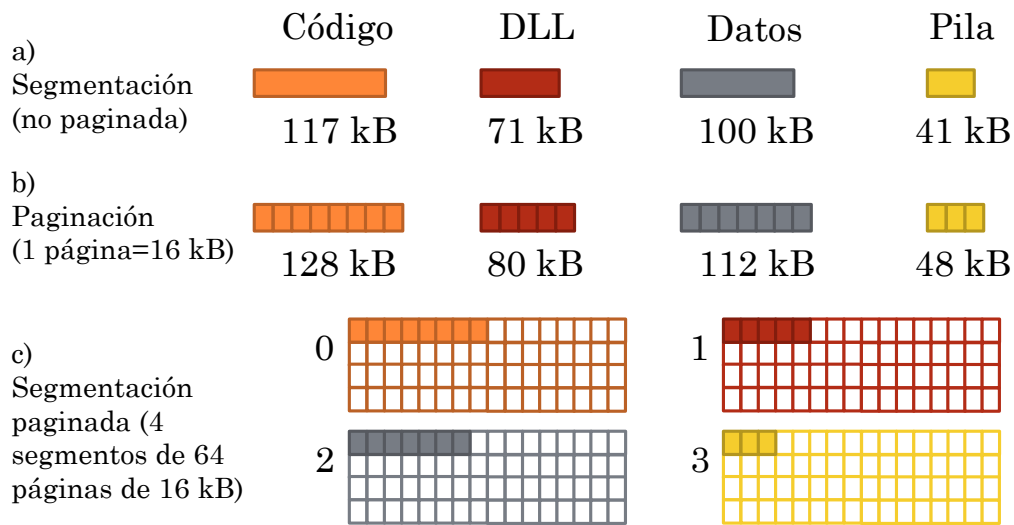
- Estos son los elementos de diseño de la memoria virtual más importantes.
 - Hay más, pero no los vamos a estudiar.

SEGMENTACIÓN O PAGINACIÓN

- **Sistema segmentado:** memoria dividida en **segmentos** de tamaño ajustado.
- **Sistema paginado:** memoria dividida en bloques o **páginas**, del mismo tamaño (potencia de 2).
- **Segmentación paginada:** se dividen los segmentos en páginas (combinación de ambas estrategias).

- Sistema segmentado:
 - La memoria se divide en bloques de tamaño variable llamados segmentos para almacenar cada uno de los objetos que necesita un proceso.
 - Este concepto de segmentación de memoria NO tiene absolutamente nada que ver con el concepto de segmentación del camino de datos estudiado en el tema 7.
 - Los segmentos se adaptan de manera natural y exacta al tamaño de los objetos del proceso (código, datos, bibliotecas de funciones, pila, etc.).
 - Esto dificulta encajar los segmentos en las zonas libres de la memoria principal.
 - Se complica también la traducción de la dirección virtual a física.
 - Es difícil garantizar que no se supera el tamaño máximo de segmento.
 - La segmentación, a diferencia de la paginación, es **VISIBLE** al programador, es decir, el programador debe estructurar adecuadamente su programa y ubicar sus partes en los segmentos adecuados.
 - Ejemplo:
 - En MIPS se indica con la directiva `.data` el segmento de las variables del programa y con la directiva `.text` el segmento de texto (código fuente).
 - En el simulador MARS el segmento de datos se carga por defecto en la dirección de memoria `0x10010000`, mientras que el de texto va a la dirección `0x00400000`.
- Sistema paginado:
 - La memoria se divide en bloques, llamados ahora “páginas”.
 - El tamaño de una página es potencia perfecta de 2 para facilitar la traducción de direcciones virtuales a físicas.
 - Las páginas de memoria virtual son asignadas a marcos de página de memoria principal.
 - Al tener un número exacto de páginas, se desperdicia parte de la última página (fragmentación interna).
 - Fácil encajar páginas en marcos de página.
 - Es más fácil garantizar que no se sobrepasa el tamaño de memoria asignado.
- Sistema segmentado paginado:
 - Híbrido.

SEGMENTACIÓN O PAGINACIÓN



10

- En este ejemplo, se muestra un proceso con cuatro objetos (código, bibliotecas DLL, datos y pila).
 - El tamaño de cada objeto es 117 kB, 71 kB, 100 kB y 41 kB, respectivamente.
- Comparación:
 - En un sistema segmentado (no paginado), los segmentos se adaptan con exactitud al tamaño de los objetos.
 - En un sistema paginado, cada objeto se divide en páginas de igual tamaño (en este caso 16 kB).
 - Como debemos tener un número exacto de páginas, se desperdicia parte de la última página, que estará en blanco.
 - Por eso el espacio total ocupado es mayor que en el caso segmentado.
 - A esta pérdida de memoria se la llama fragmentación interna.
 - En un sistema segmentado paginado, la memoria se divide en segmentos muy grandes (en este caso 4), que a su vez se dividen en páginas (en este caso de 16 kB).

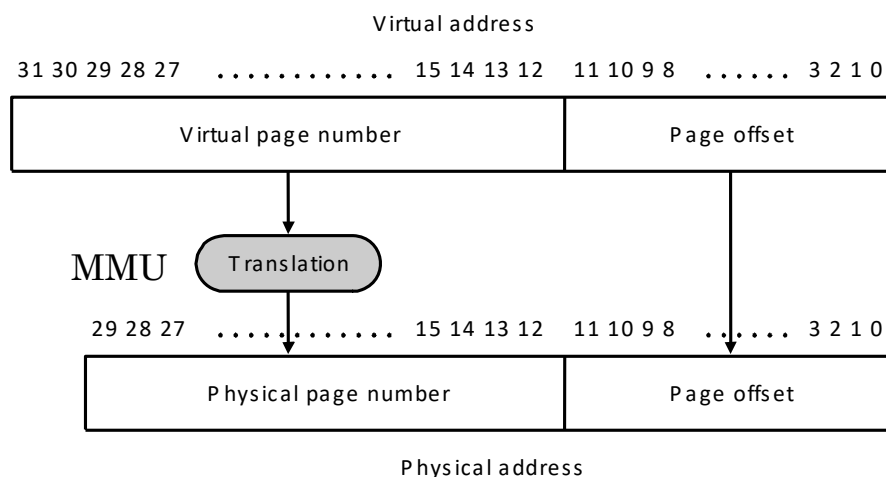
Esta figura está basada en las que aparecen en la página 467 y 468 del libro Fundamentos de Computadores de Pedro de Miguel.

ELEMENTOS DE DISEÑO DE LA MEMORIA VIRTUAL

- Segmentación o paginación
- Traducción de direcciones (identificación)
- Integración con la caché
- Otros aspectos

- Estos son los elementos de diseño de la memoria virtual más importantes.
 - Hay más, pero no los vamos a estudiar.

TRADUCCIÓN DE DIRECCIONES (POLÍTICA DE IDENTIFICACIÓN)



12

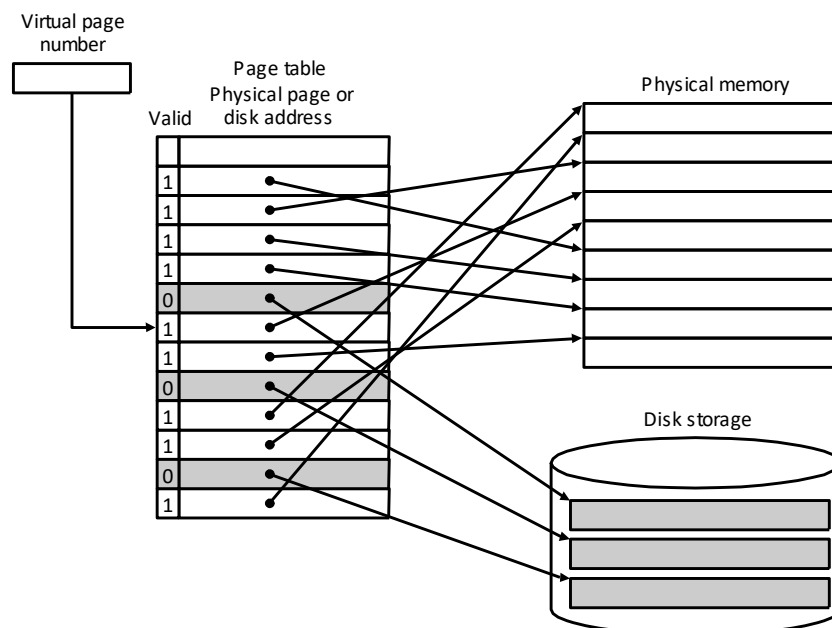
- Dada una dirección de memoria virtual, la política de identificación permite determinar si la página correspondiente está en memoria principal y, en caso afirmativo, en qué marco de página.
- En caso negativo, debe permitir averiguar en qué parte de la zona de intercambio (disco) se encuentra la página buscada.
- Esta operación se debe realizar en cada acceso a memoria y, por tanto, muy rápidamente.
 - La realiza un elemento de hardware llamado MMU o *"memory management unit"*.
- Dada una dirección de memoria virtual:
 - Los bits menos significativos representan el desplazamiento dentro de la página, es decir, identifican el byte correspondiente de una determinada palabra de una determinada página.
 - Los bits menos significativos del marco página física son exactamente iguales a los del desplazamiento de página virtual, ya que ambos tipos de página tienen el mismo tamaño.
 - Los bits más significativos corresponden al número de página virtual.
 - Se proporcionan a la MMU, que realiza la traducción a un número de página física (bits más significativos de la dirección de memoria física).
- Nótese que el número de páginas direccionables con la dirección virtual no tiene por qué ser el mismo que el número de páginas direccionables con la dirección física.
- En el ejemplo de la figura, se utilizan 12 bits para el desplazamiento de página.
 - Esto significa que las páginas (tanto físicas como virtuales) son de 4 kB = 2^{12} B de tamaño.
 - Por otro lado, el número de página virtual se expresa con 20 bits, con lo que hay 1 Mpágina virtual ($M=2^{20}$).
 - El número de página física tiene 18 bits, con lo que hay 256 kpáginas físicas ($k=2^{10}$).

TABLA DE PÁGINAS

- Las páginas se localizan usando una tabla que indexa la memoria: la **tabla de páginas**.
- Se indexa con el número de página de la dirección virtual y contiene el número de página física que le corresponde.
- Esta tabla es única por proceso. Dicho de otra manera: cada proceso tiene su propia tabla de páginas.

- Tabla de páginas (TP):
 - Estructura de datos en forma de tabla que almacena la correspondencia entre páginas virtuales y marcos de página.
 - El índice o dirección de la tabla es el número de página virtual.
 - El contenido almacenado en dicha dirección es el número de marco.
 - Su gestión y manipulación la realiza el sistema operativo.
 - Normalmente existe una tabla de página para cada proceso (programa en ejecución).
 - Su tamaño suele ser muy grande, con millones de entradas (como mínimo).
 - Por ello no es conveniente almacenarla en una memoria específica, sino que se guarda en la propia memoria principal.
 - El sistema operativo se encarga de definir la zona de memoria principal que la contiene y garantiza que “no pagina”, es decir, que permanece siempre en memoria.
 - La dirección de comienzo de la tabla en la memoria está guardada en el registro de tabla de páginas, un registro específico del procesador.

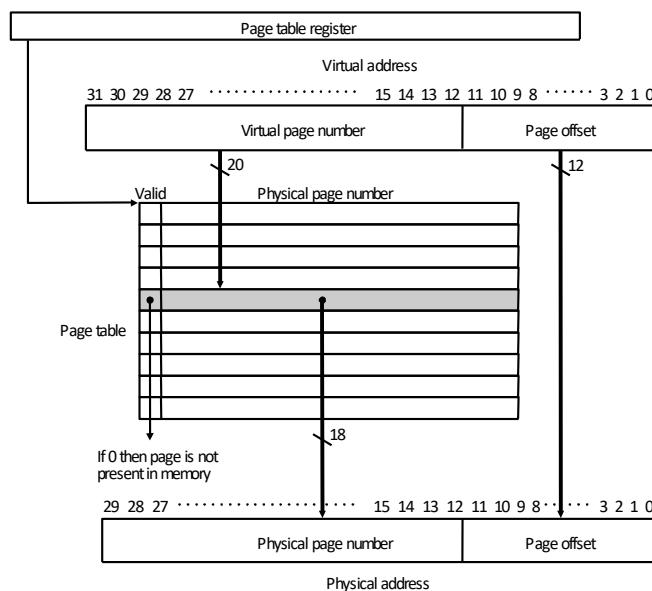
TABLA DE PÁGINAS



14

- El número de página virtual es el índice o dirección de la tabla de páginas que debemos consultar.
- En la posición apuntada encontraremos:
 - Un bit de validez o de presencia.
 - Si vale 1, la página virtual está cargada en un marco de la memoria principal (acierto de página).
 - Si vale 0, la página virtual está en el disco duro (fallo de página).
 - Un número de marco (si V=1) o la posición de la página en el disco duro (si V=0).
- En muchos sistemas, la tabla de direcciones de páginas físicas y de direcciones de páginas en disco se guarda en dos estructuras de datos separadas, aunque a nivel lógico se pueden ver como una sola.
 - Las tablas duales están justificadas en parte porque se tiene que guardar la dirección en disco de todas las páginas, aunque estén en ese momento en la memoria principal.
 - Cabe recordar que las páginas en la memoria principal y las páginas en disco tienen el mismo tamaño.

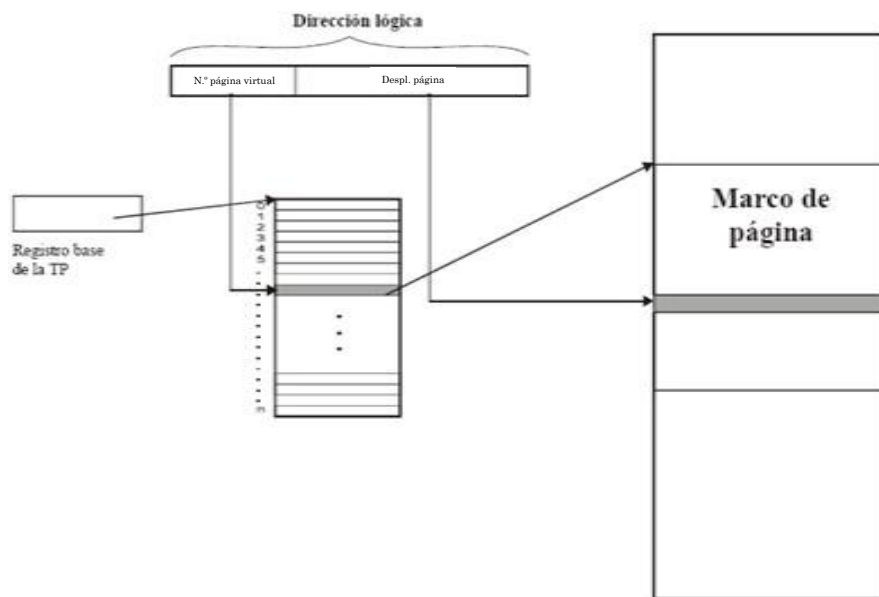
TABLA DE PÁGINAS



15

- La tabla de páginas se indexa con el número de página virtual para obtener la parte correspondiente de la dirección física.
- La dirección de memoria donde empieza la tabla de páginas se guarda en el registro de la tabla de páginas, que actúa como puntero a la tabla.
- En esta figura, el tamaño de página es de 2^{12} bytes, o 4 kB (lo sabemos por el tamaño del desplazamiento de páginas).
- El espacio de virtual de direcciones es de 2^{32} bytes, o sea, 4 GB (por el tamaño de las direcciones virtuales), mientras que el espacio físico de direcciones es de 2^{30} bytes, que permite una memoria principal de hasta 1 GB (por el tamaño de las direcciones físicas).
- El número de entradas en la tabla de páginas es entonces 2^{20} o 1 Mentradas.
- El bit de validez para cada entrada indica si la correspondencia es válida.
 - Si está apagado, la página no está presente en memoria.
- En este ejemplo, el ancho de cada entrada de la tabla de páginas es de 19 bits.
 - Es habitual redondear por exceso este tamaño a un múltiplo del byte para facilitar su acceso.
 - De 19 pasamos a 32 bits = 4 bytes.
 - Los bits extra pueden ser usados para contener información adicional, como bits de protección (RWX), etc.
- En este caso la tabla de páginas ocuparía un total de 4 MB (1 Mposiciones de 4 bytes cada una).
- Cada proceso tiene su propia tabla de página, lo cual puede ser un problema si la capacidad total de la memoria principal es del orden de las decenas de MB (caso de computadores de mitad de la década de los años 90).
- Hoy en día los computadores tienen memoria del orden de varias decenas de GB, con lo que este problema no es tan importante.

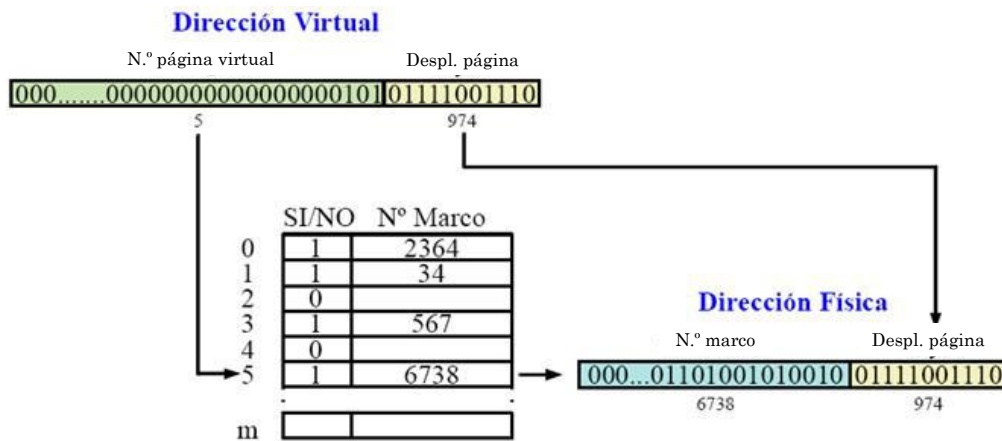
TABLA DE PÁGINAS: ESTRUCTURA DE 1 NIVEL



16

- Las tablas de página pueden organizarse en niveles.
- En la imagen se muestra una tabla de páginas de 1 nivel, que funciona como hemos explicado hasta ahora:
 - El número de página virtual (bits más significativos de la dirección virtual) es el índice o dirección de la tabla que contiene el número de marco o página física (bits más significativos de la dirección física).
 - El desplazamiento de página es el mismo tanto en la dirección virtual como en la física.
 - Basta concatenarlos al número de marco hallado en la tabla de página para conseguir la dirección física completa.

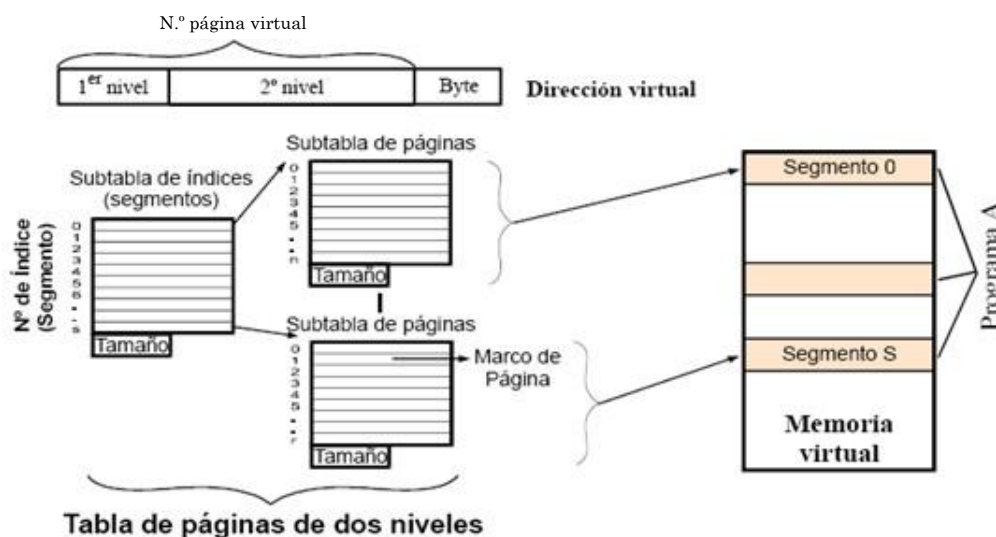
TABLA DE PÁGINAS: ESTRUCTURA DE 1 NIVEL



17

- Ejemplo:
 - El número de página virtual es en binario el 0...0101, es decir, 5 en decimal.
 - La entrada 5 de la tabla de páginas muestra un bit de validez (aquí llamado SÍ/NO) igual a 1, con lo que la página correspondiente está en un marco de página.
 - El número de marco es el 6738 (en decimal), o sea, 0001 1010 0101 0010 (binario) o 0x1A52 (hexadecimal).
 - La dirección física queda completada con el desplazamiento de página, que vale 974 (decimal), o sea, 011 1100 1110 (binario) o 0x3CE (hexadecimal).

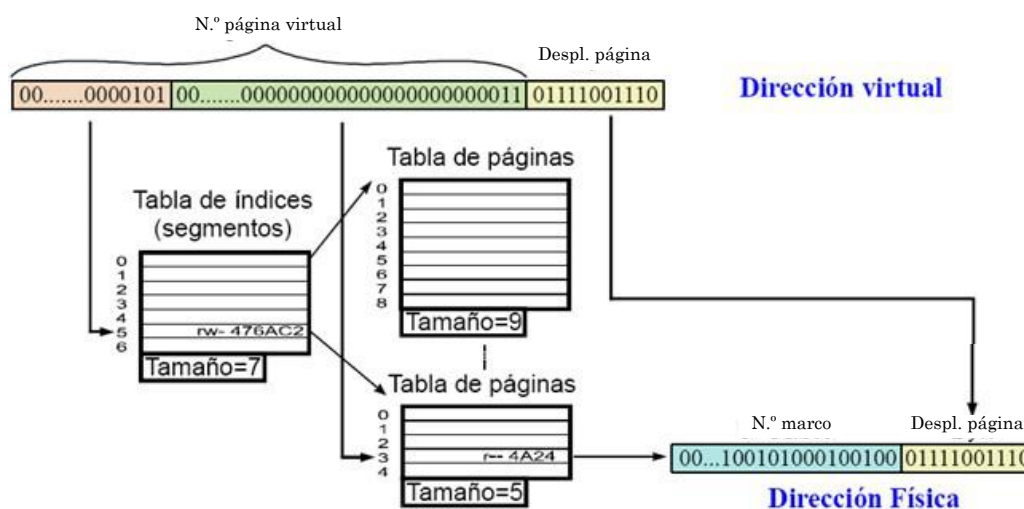
TABLA DE PÁGINAS: ESTRUCTURA DE 2 NIVELES



18

- Aquí se muestra una tabla de páginas de 2 niveles:
 - Tenemos una primera tabla (de primer nivel o tabla de segmento), cuyo índice son los bits más significativos de la dirección virtual.
 - El contenido de cada entrada de esta tabla es la dirección de memoria donde está la tabla del segundo nivel.
 - Luego por cada entrada de la tabla de primer nivel hay una tabla de segundo nivel.
 - Con la segunda parte de la dirección encontramos la posición de la tabla de segundo nivel que contiene la página física.
 - El desplazamiento de página es de nuevo igual en la dirección virtual y en la física.
 - Cada tabla almacena también su tamaño, ya que, en esta configuración, no tienen por qué ser todas las tablas igual de grandes.

TABLA DE PÁGINAS: ESTRUCTURA DE 2 NIVELES



19

- Ejemplo:
 - El número de segmento es en binario el 0...0101, es decir, 5 en decimal.
 - La entrada 5 de la tabla de primer nivel muestra un valor 0x476AC2.
 - Esta es la dirección de memoria donde está cargada la tabla de páginas de segundo nivel.
 - A continuación, leemos el índice que corresponde a la entrada de la tabla de segundo nivel.
 - En este caso vale 0...011 (binario), o sea, 3 (decimal).
 - La entrada 3 de la tabla de páginas de segundo nivel vale 0x4A24, o sea, 0...0100 1010 0010 0100 (binario), que es el número de marco.
 - La dirección física queda completada con el desplazamiento de página, que vale 974 (decimal), o sea, 011 1100 1110 (binario) o 0x3CE (hexadecimal).

TABLAS DE PÁGINAS: INFORMACIÓN

- Información almacenada en cada entrada:
 - Número del marco de página.
 - Bit de validez o de presencia, que indica si la página está o no en un marco de página.
 - Bits de protección de acceso (bits RWX, o sea, lectura, escritura y ejecución, escrito por el S. O.).
 - Bit de suciedad, para indicar que se ha modificado el contenido del marco de página.
 - Bit de acceso, útil para la política de reemplazo.
 - Bit de paginación, para indicar si la página puede expulsarse o no.
 - Bit de página cacheable, para indicarse si la información puede llevarse a la caché o no.

20

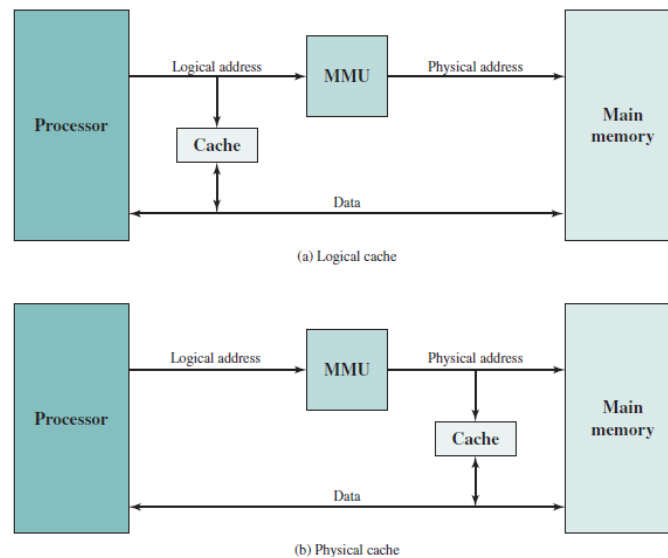
- Cada entrada de la tabla de página contiene el número de marco, más bits auxiliares (de presencia, de protección, de suciedad, etc.).
- Nótese que en la tabla de páginas NO se necesitan etiquetas porque NO es una caché.
- El número de página virtual TAMPOCO se almacena, ya que es solo el índice de la tabla.

ELEMENTOS DE DISEÑO DE LA MEMORIA VIRTUAL

- Segmentación o paginación
- Traducción de direcciones (identificación)
- Integración con la caché
- Otros aspectos

- Estos son los elementos de diseño de la memoria virtual más importantes.
 - Hay más, pero no los vamos a estudiar.

INTEGRACIÓN CON LA CACHE



22

- Sumando la memoria caché al sistema de memoria virtual, la caché puede ubicarse antes o después de la traducción de la dirección virtual realizada por la MMU.
 - Si está antes, la caché trabaja con direcciones virtuales.
 - Si está después, trabaja con direcciones físicas.
 - En este caso, el tiempo de acceso es la suma del tiempo de traducción más el tiempo de la caché.
- Con cachés multinivel, se suele simultaneizar el acceso a la caché L1 y a la MMU.
 - Para los niveles inferiores de caché hay que esperar a la traducción de la dirección para comenzar la operación.

Figura extraída del libro William Stallings, Computer Organization and Architecture, 9.ª edición, pág. 124, 2013.

ELEMENTOS DE DISEÑO DE LA MEMORIA VIRTUAL

- Segmentación o paginación
- Traducción de direcciones (identificación)
- Integración con la caché
- Otros aspectos

- Estos son los elementos de diseño de la memoria virtual más importantes.
 - Hay más, pero no los vamos a estudiar.

OTROS ASPECTOS

- Política de extracción: bajo demanda
- Función de correspondencia: totalmente asociativa
- Algoritmo de reemplazo: LRU
- Política de escritura: *write-back*

- Política de extracción:
 - Decide qué páginas son migradas de la zona de intercambio a marcos de página.
 - Tarea realizada por el sistema operativo.
 - Generalmente se utiliza una paginación por demanda, es decir, se envía una página solamente cuando se necesita y da un fallo de acceso.
- Función de correspondencia (o política de ubicación):
 - Establece es a qué marco de memoria principal se asigna una página de memoria virtual.
 - Se usa totalmente asociativa:
 - Podremos ubicar cualquier página virtual en cualquier marco de página.
 - Tarea realizada por el sistema operativo.
- Algoritmo de reemplazo:
 - Establece qué marco de memoria principal debe vaciarse y devolverse a disco cuando ya no hay hueco para traer una nueva página (que ha dado fallo).
 - Se elige el marco usado menos recientemente (LRU).
 - Necesitamos bits de uso reciente para cada marco de la memoria principal.
 - Tipos de reemplazo:
 - General:
 - La página que vamos a eliminar puede pertenecer a otro proceso distinto del que originó el fallo.
 - Individual:
 - La página que vamos a eliminar debe pertenecer al mismo proceso.
 - Hiperpaginación (*thrashing*):
 - Fenómeno consistente en dedicar la mayor parte del tiempo a intercambiar páginas en vez de a ejecutar instrucciones.
 - El algoritmo de reemplazo intenta predecir qué páginas tienen menos probabilidad de usarse en el futuro cercano, según el uso reciente.
- Política de escritura:
 - Debido a la gran diferencia de velocidades entre la memoria principal y el disco duro, se prefiere la escritura retardada o *write-back*.
 - Las escrituras se realizan solo en los marcos cargados en memoria principal.
 - Cuando un marco sea sustituido por otro, se realiza la copia en el disco duro.
 - Necesitamos un bit de suciedad para cada marco de la memoria principal.

CONTENIDOS

- Introducción
- Elementos de diseño
- Aumento del rendimiento: memoria TLB

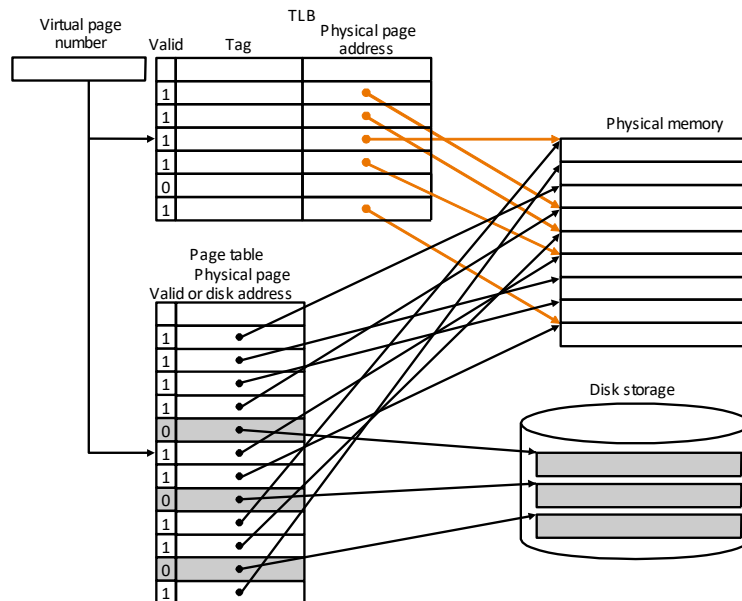
AUMENTO DEL RENDIMIENTO: MEMORIA TLB

- **TLB** (buffer de traducción de direcciones o “*Translation-Lookaside Buffer*”): memoria caché especial para acceder más rápidamente a la tabla de páginas.
 - Tamaño pequeño.
 - Totalmente asociativa.
- La TLB contiene solo la información correspondiente a los últimos accesos a la tabla de páginas:
 - Número de página virtual.
 - Número de marco.
- Puede haber bits adicionales como los de uso reciente.

26

- La tabla de páginas está en memoria principal:
 - Si cada vez que queremos acceder a un dato o instrucción en memoria tenemos que hacer un acceso a memoria extra para leer la tabla de páginas para hallar su posición física, ¿qué sentido tiene todo?
- La memoria TLB:
 - Permite acelerar el acceso a la memoria porque se construye con la misma tecnología que la caché (más rápida que la memoria principal).
 - Se basa en la localidad de la tabla de páginas:
 - Cuando se hace una traducción para un número de página virtual, probablemente sea necesaria en breve porque las referencias a las palabras de esa página tienen localidad espacial y temporal.
 - La TLB almacena los últimos accesos a la tabla de páginas: en qué marco están las últimas páginas virtuales consultadas.
 - Lo primero de todo se consulta la TLB.
 - Solo se accede a la tabla de páginas en caso de fallo de TLB.

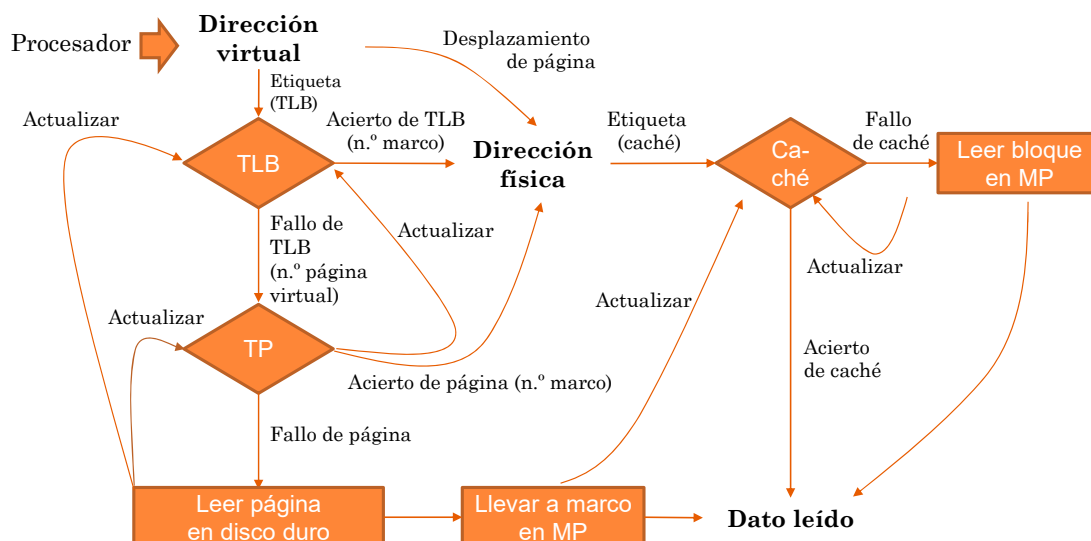
AUMENTO DEL RENDIMIENTO: MEMORIA TLB



27

- La TLB funciona como una caché de la tabla de páginas para las entradas que están asignadas a páginas físicas.
 - Contiene un subconjunto de las correspondencias entre páginas virtuales y páginas físicas que están en la tabla de páginas.
 - Como la TLB es una caché, necesita tener una etiqueta en cada entrada (el número de página virtual).
 - Si no hay ninguna entrada que corresponda a una página, se produce un fallo de TLB y se tiene que examinar la tabla de páginas.
 - La tabla de páginas suministra el número de página física para la página (que luego se puede usar para construir la entrada en la TLB) o indica que la página está en el disco, en cuyo caso ocurre un fallo de página.
 - Como la tabla de páginas tiene una entrada para cada página virtual (recordemos que no es una caché), no precisa de ninguna etiqueta.

AUMENTO DEL RENDIMIENTO: MEMORIA TLB



28

- El diagrama muestra la secuencia de acciones en el caso de tener una memoria virtual con TLB y tabla de páginas (TP) superpuesta a una memoria caché (que admite direcciones físicas, no lógicas) más la memoria principal más un disco duro.
 - En particular, se trata de una lectura (se deja como ejercicio pensar qué ocurriría en una escritura).
- Pasos:
 - Primero, el procesador publicaría una dirección virtual en el bus de direcciones.
 - El primer nivel al que se recurre es la TLB.
 - La dirección virtual se descompone en campos, de entre los cuales destaca la etiqueta (número de página virtual).
 - La TLB se recorre entera para buscar si alguna entrada tiene una coincidencia con esta etiqueta.
 - En caso de haberla, se trata de un **acierto de TLB**.
 - Leer dicha entrada proporciona directamente el número de marco y se puede calcular la dirección física.
 - En caso de **fallo de TLB**, habrá que acceder a continuación a la TP.
 - Supongamos fallo de TLB:
 - La dirección virtual se descompone en los campos de número de página virtual y desplazamiento de página.
 - En la TP, se localiza la entrada con índice igual al número de página virtual y se comprueba el bit de validez.
 - En caso de ser 1, se trata de un **acierto de página**.
 - Dicha entrada nos da directamente el número de marco y se puede calcular la dirección física.
 - En caso de que el bit valga 0, se trata de un **fallo de página**.
 - La página que buscamos no está en la memoria principal (MP).
 - Debemos ir al disco duro a leer la página correspondiente y llevarla al marco de la MP que corresponda.
 - Una vez leído el dato, debemos actualizar tanto la TP (para anotar en qué marco de MP hemos almacenado dicha página), como la TLB (para anotar el último acceso a la TP).
 - En los casos de acierto de TLB o acierto de página, disponemos ya del número de marco o página física.
 - Junto con el desplazamiento de página que proviene de la dirección virtual, podemos construir la dirección física, que se proporciona a la memoria caché.
 - La dirección física se descompone en campos, uno de los cuales es la etiqueta.
 - La caché habrá que recorrerla para buscar coincidencia con dicha etiqueta.
 - Si alguna línea de caché la contiene, se trata de un **acierto de caché**.
 - En ese caso, esa entrada contiene el bloque en el que está el dato que queríamos.
 - En caso de no haber coincidencia, se trata de un **fallo de caché**.
 - Si ocurre esto, habrá que ir a la MP para leer el bloque que contiene el dato y almacenarlo en la caché en la línea que corresponda.
 - A la vez, ya disponemos del dato que queríamos.